



Internship at Visteon

Name	CDS id	Mentor
Eepshita Pahari	epahari	Soumya V
Poojapriya S	psenth10	Naveen Madala
Pushkar Paul	ppaul3	Harshvardhan Patil
Rahul Hegde	rhegde2	Sanket Chaudhari

Manager: Md Irshad

[visteon.com](https://www.visteon.com)

Visteon®

VHAL

visteon.com



CONTENTS

1. Introduction.
2. Daily Tasks and Responsibilities.
3. Tools/Technologies/Methodologies Used.
4. Skills And Knowledge Gained.
5. Individual Learnings & Technical Exposure.
6. Reflections & Learnings.
7. Conclusion.

Introduction

We are part of the Vehicle Hardware Abstraction Layer (VHAL) team under the Android Open-Source Project (AOSP) domain.

- This team focuses on providing a standardized interface for Android Automotive OS (AAOS) to communicate with the vehicle's underlying hardware components and ECUs.
- The VHAL team is responsible for implementing and maintaining the VHAL service, which involves defining vehicle properties, handling property requests (read/write), and managing property subscriptions.
- They ensure the robust and efficient flow of data between the Android framework and the vehicle's hardware.
- We got exposure to various topics under AOSP and its source code, its folder structure, its different components, and more.

Daily tasks and responsibilities

January and February

- We completed online courses assigned in cornerstone covering:
 - Company and Product Overview
 - Behavioral Curriculum
- Participated in in-person training sessions on:
 - Automotive Product Overview
 - UML Patterns
 - Automotive Networks
 - Embedded C for Automotive Industry
 - Automotive Coding Standards
 - System Validation Training
- Completed a series of Embedded C assignments on Raspberry Pi.
- We honed our skills in various C++ and advanced C++ topics like Pointers, OOPs, Multi-threading, files, exception handling, etc.

Daily tasks and responsibilities

March and April

- Actively took part in C Language Training followed by the GEAR UP – C Hackathon and applied core programming concepts to solve real-world problems and enhance coding proficiency.
- Went through VHAL documentation to get a better understanding of how Android Automotive communicates with vehicle hardware, including how different vehicle properties are managed and accessed.
- Completed ASPICE Overview Training and VBOS/VPDS/Vistway Overview sessions, gaining expertise in automotive software development standards and vehicle platform architectures.
- Implemented the JSON File and Excel Sheet Comparison task.
- Successfully synced Android 14 source code and explored variety of adb commands when interacting with android devices.

Daily tasks and responsibilities

May and June

- Completed C++ training and participated in the FLEETXPRESS – C++ Hackathon, applying OOPS concepts through hands-on tasks and collaborative coding challenges.
- Attended problem-solving sessions to strengthen logical thinking and improve approach to coding challenges.
- Learned and prepared weekly presentations on topics:
 - Android Boot Sequence
 - HAL
 - AIDL
 - Power Management

to build a better understanding of Android system components.

Tools/Technologies/Methodologies Used

- **Tools:**

- Ubuntu/Linux Terminal – For compilation, flashing and script execution.
- WSL [Windows Subsystem for Linux] - Occasionally used for compiling C/C++ Code.
- Git – Syncing with code repositories.
- Android Studio – For application development.

- **Technology:**

- Build System – lunch, make, Soong/Ninja.
- Device Flashing – adb.

- **Methodology:**

- Agile Development – Task tracking.
- Debugging – logcat, adb shell.

Skills and Knowledge Gained

Technical Skills

- Developing and testing Android Application
- Using Command Line tools for System Debugging and Development
- Managing Source Code with Git
- Automating builds and flashing images to target devices

Skills and Knowledge Gained

Soft Skills

- Team Collaboration – Working with team members and mentors from around the country
- Effective Communication – Reporting progress, asking clarifying questions, and presenting ideas
- Problem Solving – Debugging issues and tracing errors
- Adaptability – Learning new tools and concepts quickly in a dynamic environment
- Professional Etiquette

Individual Learning and Technical Exposure

Eepshita Pahari [EPAHARI]

- **Embedded Systems Experience:** Acquired hands-on knowledge of device flashing and device tree configurations. Successfully flashed Tata Skylark device using SP Tool, demonstrating practical embedded systems capabilities.
- **C++ Fundamentals and OOP Concept:** Built simple running codes to implement the topics studied, apart from the C++ tools built.
- **AOSP Team Collaboration:** Participated in daily technical sessions with Kolkata AOSP team, developing proficiency in:
 - Java programming fundamentals
 - CMake build system configuration
 - Zero MQ messaging implementation
 - Advanced log analysis techniques
- **Ongoing Task:**
 - Implementing modular HAL interfaces using HIDL and Soong .bp files.
 - Studying remote error reporting, OTA updates, and diagnostics in automotive systems. Focused on integrating Android Automotive services with secure cloud communication.

Individual Learning and Technical Exposure

Eepshita Pahari [EPAHARI]

Android Application Building:

- **Core Android Development**
 - Built application to understand Activity Life Cycle - Understood fundamental Android component behavior
 - Calculator application to implement functionalities of constraints and functions - Developed practical experience with UI layouts and mathematical operations
 - Tip Calculator application to try hands on different design styles - explored diverse visual approaches and user interface design patterns
- **Intra and Inter Application Communication**
 - Message sender from one activity to another activity of the same application - Implemented internal activity communication using Intent mechanisms
 - Message Sender and Message Receiver – Two different applications to receive and transmit message
- **Game Development** - Bigger number game to combine design and implementation - successfully integrated aesthetic user interface with functional game logic

Individual Learning & Technical Exposure

Poojapriya S [PSENTH10]

- Mahindra W616 Flashing - Flashing involves updating the firmware of a vehicle's electronic control units (ECUs) using QFIL(Qualcomm Flash Loader) to understand how embedded automotive systems are updated.
- JLR Flashing, Testing – Gained hands-on experience while testing services like EndofLine. Supported to verify that the services were functioning as intended after flashing, improving understanding of software integration in automotive systems.
- Analyzed key logs from vstgloballogs such as system, event, crash, and kernel logs to during testing and flashing which helped in understanding how different types of logs contribute to troubleshooting and validating software behavior in automotive systems.
- Learned various adb commands, dmesg and logcat commands during testing.
- Also collaborated with Pre-int and HMI when testing domain system changes.
- Researching about different strategies used for crash event detection, including both polling and heartbeat (push) mechanisms.

Individual Learning & Technical Exposure

Poojapriya S [PSENTH10]

- Polling is a method where a system repeatedly checks at regular intervals if a new event (e.g., crash) has occurred.
- Heartbeat is a system where an event (like a crash) automatically notifies another component without needing periodic checks.

Implemented CPP compiler-level program

- visualized how smart pointers manage memory (allocation and deallocation)

Individual Learning & Technical Exposure

Pushkar Paul(PPAUL3)

Game Development: Developed a series of game using Ray-Lib Library for development of Games like Tic-Tac-Toe, Snake Game, Tetris, Ping-Pong.

Android 14: Build Source code and used it to compile and run cpp program in the adb shell.

AOSP team IN-Person Meeting: Participated in meeting in-person with AOSP team for developing proficiency in different aspects like:

- Java Programming Fundamentals.
- Fstab, mount, Android Folder Structure.
- Zero-MQ messaging Service.
- Advance log analysis technique.

SOONG AIDL Implementation: Implementing the AIDL interfacing which uses services for a Calculator along with the Server and the Client.

Android App Development:

- Implemented Android Interface Definition Language (AIDL) to enable inter-process communication (IPC) between a server and client application in Android Studio, facilitating structured and secure data exchange across application boundaries.
- Created FlashLight Application for voltage and SOS Message Regulating through flashlight.

Individual Learning & Technical Exposure

Pushkar Paul(PPAUL3)

Ongoing tasks:

- Implementing Deamon on the calculator so that the service starts on-boot.
- Implementing the Some-ip for communicating with two services in soong.
- Implementing Flashlight through soong for voltage and disco in led with the pico.
- Device Management R&D :
 - **logd / logcatd** – System-level logging daemons collecting logs from all sources including native crashes.
 - **liblog (__android_log_print)** – Native C/C++ logging API used for custom crash messages.
 - **Systrace / atrace** – Legacy tracing tools capturing system stalls and slow operations (useful for ANR diagnosis).
 - **SELinux Audit Logs** – Detect permission-denied events that can lead to service or app crashes.
 - **dmesg / printk** – Kernel ring buffer logs that reveal low-level crashes, panics, and hardware watchdog resets.
 - **/proc and /sys Filesystems** – Monitor per-process crash signals, memory leaks, and thread states.
 - **Watchdog Daemons (Software/Hardware)** – Automatically detect and reboot the system on hangs or freezes.
 - **KCrash (Linux/Qt)** – Crash handler for embedded or Qt-based applications (useful in IVI systems).
 - **Android Health HAL / VHAL (Automotive)** – Reports hardware/software health, can signal crash-like faults in components.
 - **ACRA (App Crash Reporter Library)** – Java-layer crash report collector that uploads crash logs to remote servers.
 - **tombstoned intercept tools** – Low-level mechanism to hook into native crash dumps for advanced crash tracking.
 - **GDBServer** – Debug remote native services post-crash via TCP, useful for embedded environments.
 - **LLDB (NDK Debugger)** – Debug native crashes in Android Studio using breakpoints and memory inspection.

Individual Learning & Technical Exposure

- **Rahul Hegde [rhegde2]**
- **Automotive Standards:** Attended training session on Agile Training: Scrum Practices assigned by my mentor.
- **Hardware Flashing & Debugging:** Gained hands-on experience in device flashing, specifically learning how to flash a Tata Skylark device using the SP Flash Tool.
- **Android Inter-Process Communication (IPC) Implementation using AIDL:** Implemented an Android IPC mechanism in Android Studio using AIDL to enable secure communication between a Client and a Server application via a bound service.
 - Created two apps: Server (AIDLService) & Client.
 - Defined AIDL interface IVehicleService.aidl to return speed for vehicle type
 - Implemented a bound service in server using IVehicleService.Stub
 - Added custom permission for secure binding
 - Used Intent with setPackage() for explicit binding
 - Debugged using Logcat to trace service binding & method calls
 - Verified output on physical device — successfully retrieved speed

Individual Learning & Technical Exposure

Ongoing task: Diag and Debug Research Summary

- Crash Types Monitored
 - Java app crashes (uncaught exceptions)
 - Native crashes (signals like SIGSEGV)
 - Service crashes
 - ANRs (Application Not Responding)
 - Kernel crashes & system resets
- Key Components Detecting Crashes
 - ActivityManagerService (AMS): Detects app/service crashes & ANRs
 - debuggerd daemon: Captures native crash dumps (tombstones)
 - Linux Kernel: Sends signals for native crashes, detects kernel panics
 - Watchdog.java: thread monitors and detects system server crash.
- Crash Event Generation & Logging
 - AMS methods: appNotResponding(), handleAppCrash() trigger events
 - debuggerd generates native crash reports
 - DropBoxManagerService stores crash/ANR reports
 - Logs aggregated in logcat for user/developer access
- Next Steps
 - Study AMS, debuggerd, DropBoxManagerService source code
 - Use APIs to listen for crash events and retrieve logs

Reflections and Learning

Understanding of corporate work culture

Learnt how to work in the professional environment by following the step by step procedures and collaborating with teams and mentors.

Exposure to the development in AOSP Domain

Gained Hands on experience in app development and AOSP code implementation along with the bp files and folder structure and implementing the client and server application.

Improved communication and collaboration skills

Developed a confidence in sharing the ideas, asking questions , working with diverse team members, which helped build strong interpersonal and collaboration skills.

Professional Growth and handling Responsibilities

Gained experience to handle the responsibilities and adapting to new challenges and undersatnding how to contribute effectively in corporate projects.

Conclusion

We extend our sincere gratitude to the Manager and Mentors for their unwavering support and invaluable guidance throughout our internship journey. The hands-on experience and knowledge we have acquired have equipped us to face future challenges in the industry with confidence. We look forward to applying these skills and insights in our forthcoming endeavors and contributing meaningfully to innovative projects.

THANK YOU